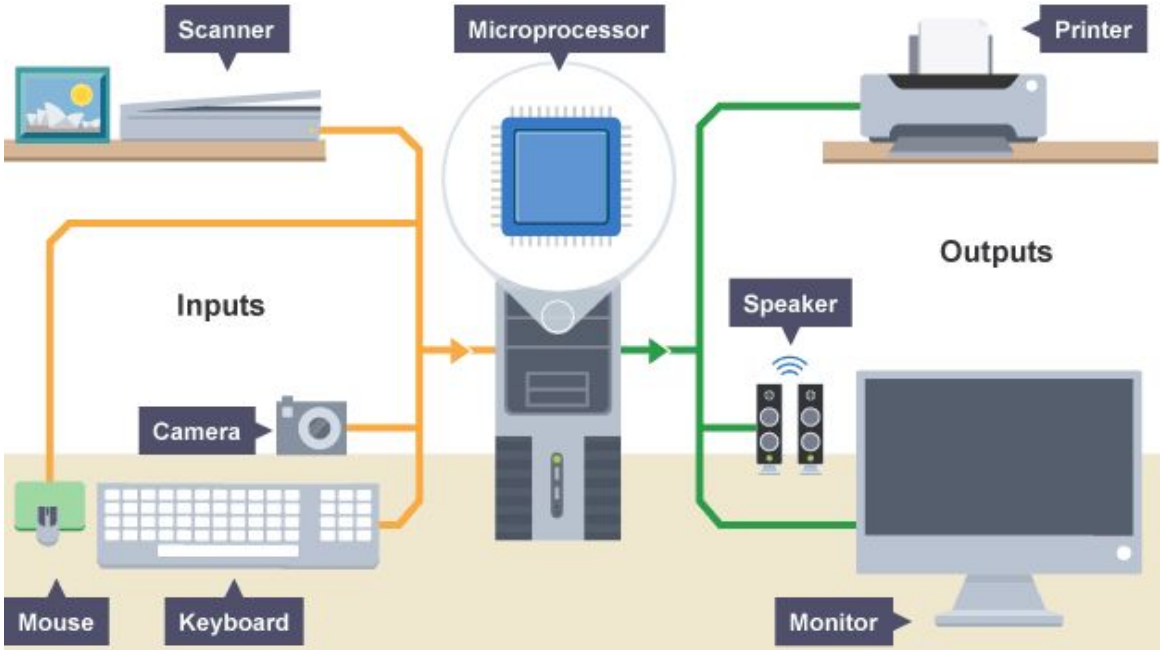
CSE 4305
Computer Organization and Architecture

Lecture 7
Input/Output

Sabrina Islam
Lecturer, CSE, IUT
Contact: +8801832239897
E-mail: sabrinaislam22@iut-dhaka.edu

Input-Output Devices



Input-Output Devices

We can broadly classify external devices into three categories:

- **Human readable:** Suitable for communicating with the computer user.
  - E.g.: video display terminals and printers.
- **Machine readable:** Suitable for communicating with equipment.
  - E.g.: magnetic disks, tape systems, sensors and actuators.
- **Communication:** Suitable for communicating with remote devices.
  - E.g.: wifi, modem.

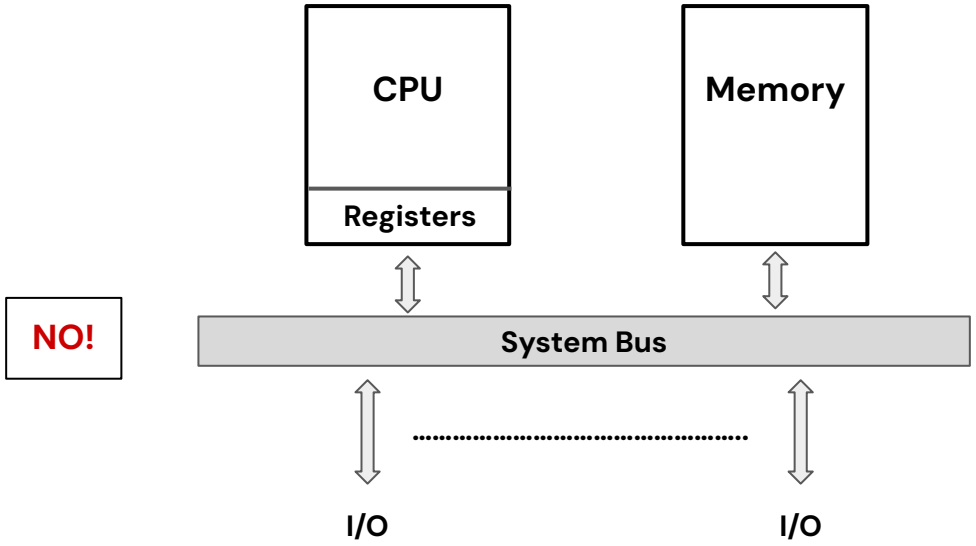
Input-Output Devices

Different devices have different requirements:

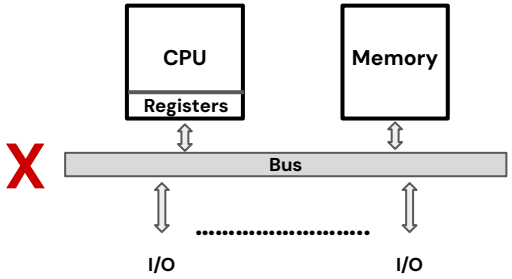
- Peripheral logic
- Peripheral speed
- Peripheral data protocol

Input-Output Organization

- How can the communication between processor and external devices be performed?
- Can we directly connect I/O devices to the system bus?



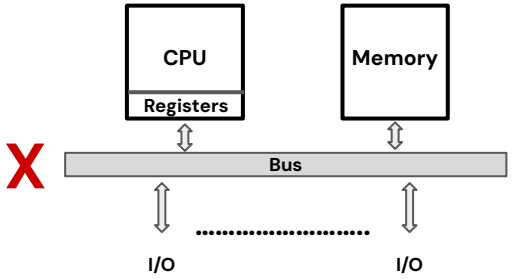
Input-Output Organization



We cannot directly connect I/O devices to system bus, because

1. **There are a wide variety of peripherals:**
   a. Each with its method of operation.
   b. Impractical to incorporate diverse set of logic within the processor.
2. **Data transfer rates**:
   a. Peripherals often operate much slower than processor speed or memory.
   b. Impractical to use the high-speed system bus to communicate directly.
   c. Transfer rate may also be faster than that of the memory/processor.
   d. Mismatch would lead to inefficiencies if not managed properly.

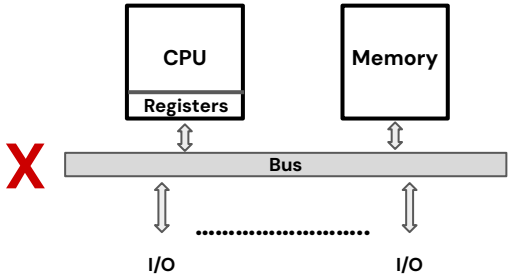
Input-Output Organization



We cannot directly connect I/O devices to system bus, because

3. **Data formats:**
   a. Peripherals often use different communication protocol:
      i) data format not necessarily the same as the computer.
      ii) word length not necessarily the same as the computer.
   b. The processor would have to convert these back-and-forth

Input-Output Organization



So we come to the conclusion that:

- Processor speed is usually faster than peripherals.
- No need to slow down the processor to do interaction with I/O sources.
- **What can be done to solve this problem?**
  - Create a separate entity responsible for managing I/O sources:
    - Manage the communication with the peripherals.
    - Store the data input / output from and to peripherals.
    - Detect errors.
  - **It is called the I/O module!**

I/O Module

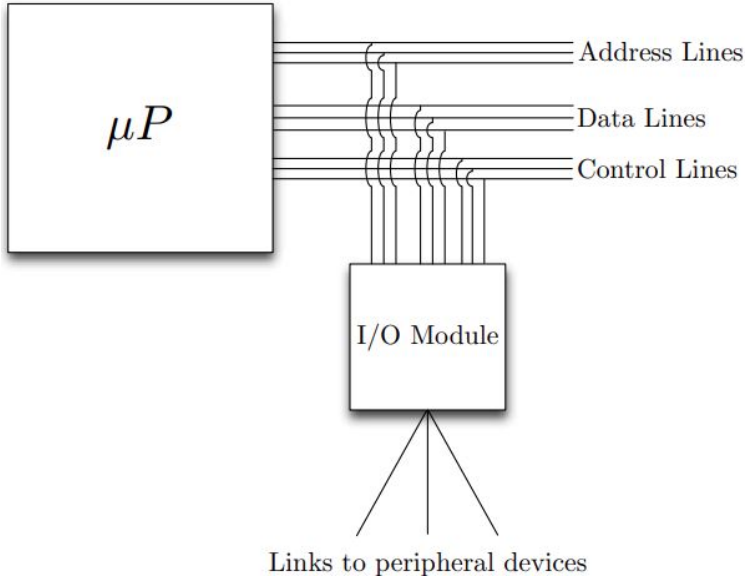


Fig: Generic Model of an I/O Module.

Main components of an external device

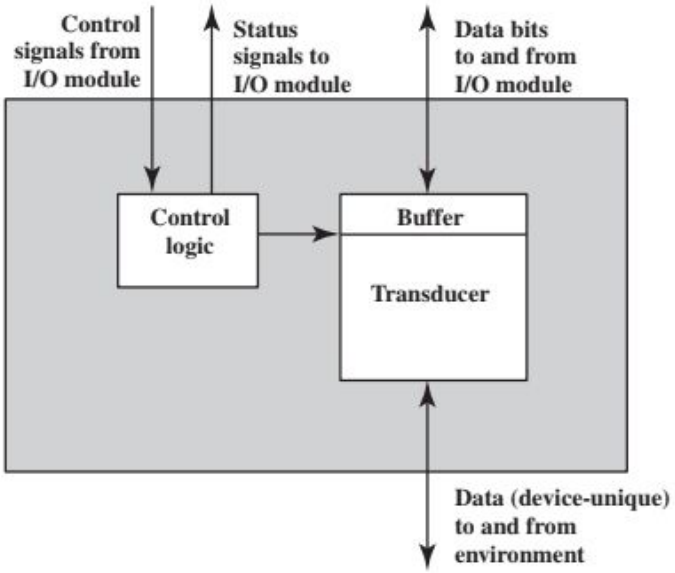


Fig: Block Diagram of an External Device.

Main components of an external device

- **Control Signals:** To determine the function the device will perform, e.g.:
  - Send data to the I/O module
  - Accept data from the I/O module
  - Report status
  - Perform function particular to the device (e.g.: position a disk head)
- **Data signals:**
  - Set of bits to be sent to or received from the I/O module.
- **Status signals:** To indicate the state of the device, e.g.:
  - READY/NOT-READY to show whether the device is ready for data transfer
- **Control logic:**
  - Controls the device in response to direction from the I/O module.
- **Transducer:** Converts data
  - From electrical to other forms of energy during output.
  - And from other forms to electrical during input.
- **Buffer:**
  - To hold data.

Major functions of an I/O module

- **Control and timing:**
  - At any time the processor may communicate with external devices in unpredictable patterns. This depends on the program's need for I/O.
  - Thus, the I/O function includes a control and timing requirement to coordinate flow of traffic between resources.
- **Processor communication:**
  - **Command decoding:** I/O module accepts commands from processor and decodes. For example, I/O module commands for a disk drive: READ sector, WRITE sector, SEEK track number and SCAN sector ID.
  - **Data:** Exchanged between the processor and the I/O module (data bus).
  - **Status reporting:** Important to know the status of the I/O module. Sometimes I/O module may not be ready because of working on a previous I/O command. This needs to be reported with a status signal.
  - **Address recognition:** I/O module has one unique address for each peripheral it controls.

Major functions of an I/O module

- **Device communication**
  - 1. Commands, 2. Status information, 3. Data
- **Data buffering:** There is a high transfer rate between main memory and processor but it is lower for the external devices.
  - Data sent from Main memory to an I/O module are buffered in the module, then sent to peripheral device at a data rate the device can sustain.
  - Data sent from I/O module to main memory:
    - Data are buffered from peripheral. This is done so as not to tie up the memory in a slow transfer operation.
    - Also If the I/O device operates at a rate higher than the memory access rate, I/O module performs the needed buffering operation.
- **Error correction:**
  - Mechanical and electrical malfunctions reported by the device.
    - E.g.: paper jam, bad disk track
  - Unintentional bit changes during device transmission.

I/O module structure

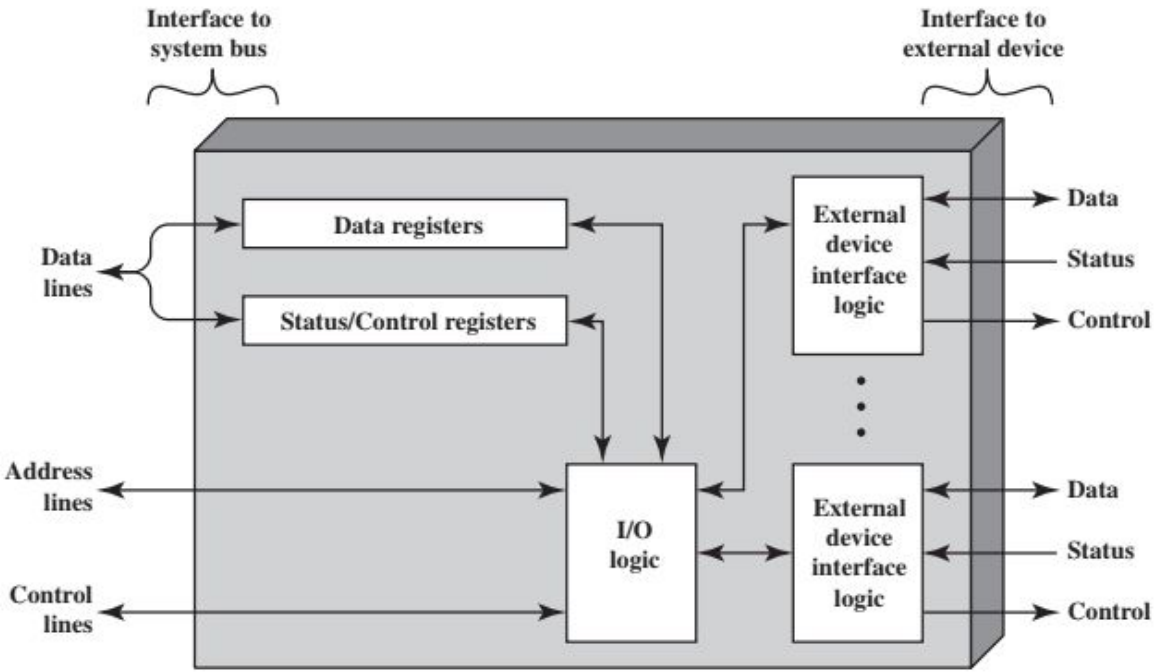


Fig: Block Diagram of an I/O module.

Organization of a generic I/O module

- Module connects to computer through a set of signal lines.
- Data transferred to and from the module are buffered in data registers.
- Status registers provide status information, also function as control registers to accept processor control info.
- Logic within the module interacts with the processor via control lines:
  - Processor uses the control lines to issue commands to the I/O module.
  - Some of the control lines may be used by the I/O module, e.g. arbitration and status signals.
- Module must also be able to recognize and generate addresses for each device it controls.
- I/O module contains logic specific for a set of interfaces.

I/O commands

To execute an I/O-related instruction, Processor needs to issue the following:

- An address specifying the particular I/O module and external device.
- An I/O command which can be of the following type:
  - Control, Test, Read and Write

I/O command type

- **Control**: Used to activate a peripheral and tell it what to do
  - e.g Rewind magnetic-tape;
- **Test**: Tests I/O module and its peripherals.
  - Is the peripheral powered on?
  - Is the peripheral available for use?
  - Has the most recent I/O operation completed? Did any errors occur?
- **Read**: I/O module obtains a data item from the peripheral:
  - Processor requests I/O module to place data on bus (the data is placed in an internal buffer).
- **Write**: I/O module writes a data item to the peripheral:
  - I/O module reads data from bus.
  - I/O module transmits data to peripheral.

I/O Transfer modes

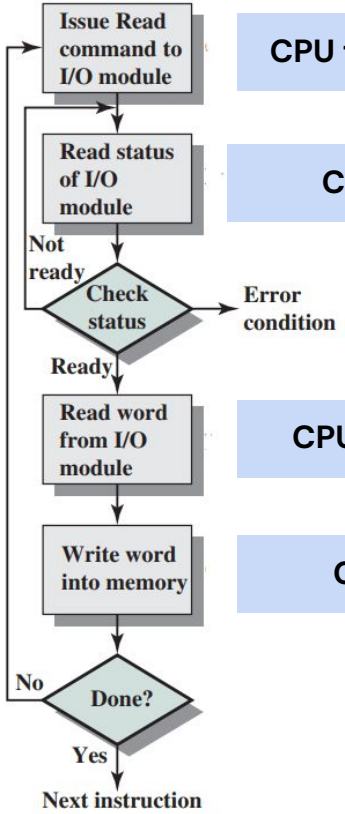
- Programmed I/O
  - I/O-to-memory transfer through processor.
- Interrupt-Driven I/O
  - I/O-to-memory transfer through processor.
- Direct Memory Access (DMA)
  - Direct I/O-to-memory transfer.

Programmed I/O

When a program is in execution, if an I/O related instruction is encountered

- The processor issues a command to the appropriate I/O module
- The I/O module will perform the requested action and and then set the appropriate bits in the I/O status register.
- Processor is **not interrupted** by the I/O module after it is ready, so
    - processor periodically checks the status of the I/O module.

Programmed I/O



CPU to I/O module

CPU reads

Error occurred while performing the task

CPU from I/O module

CPU to memory

Programmed I/O

Disadvantages:

- The processor has to wait a long time for the I/O module.
- The processor, while waiting, must repeatedly interrogate the status of the I/O module.
- If processor is faster than I/O module: wasteful of processor time.
- The performance of the entire system is severely degraded.

Interrupt-driven I/O

When a program is in execution, if an I/O related instruction is encountered

- After issuing a command to the I/O module, the processor returns to perform other useful works instead of waiting.
- Processor is **interrupted** when task is done by the I/O module.

Interrupt-driven I/O

From the point of view of the **I/O module**:

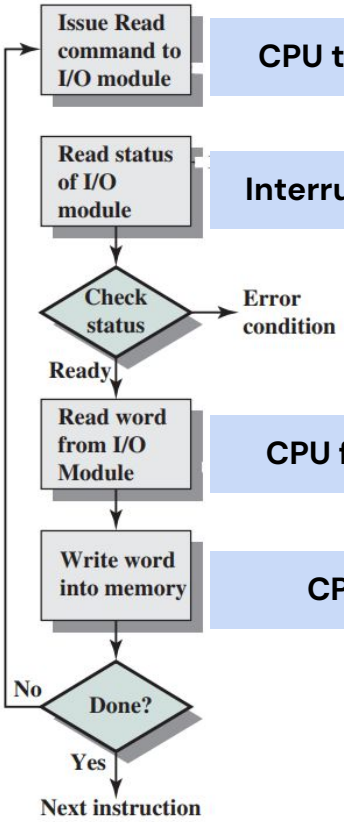
- Module waits for processor to request data
- When request is made:
  - Module interacts with peripheral
  - Once the data is completely buffered, data are place on data bus
- An interrupt signal is sent to the processor over a control line
- Module becomes available for another I/O operation.

Interrupt-driven I/O

From the point of view of the **processor**:

- A READ command is issued to I/O module
- Processor goes off to do something else
- Processor checks for interrupts at the end of each instruction cycle
- When the interrupt from the I/O module occurs
  - processor saves program context
  - processor proceeds to read data from I/O module
  - processor stores data in memory
- Processor then restores previous program context
- Processor resumes execution of previous program

Interrupt-driven I/O



CPU to I/O module, continues other work

Interrupt occurs and CPU reads the status

Error occurred while performing the task

CPU from I/O module

CPU to memory

Disadvantage??
(For each transfer of data it involves the processor)

Interruption-based design issues

Issue 1: How does the processor determine which device issued the interrupt?

Solutions:

1. **Multiple interrupt lines:**
   - The most straightforward approach but it is impractical to dedicate more than a few bus lines to interrupt lines
   - Consequently, it is likely that each line will have multiple I/O modules attached to it, so not very practical.
2. **Software poll:**
   - Processor calls an interrupt routine that polls each I/O module. The I/O module responds positively if it set the interrupt.
   - Disadvantage: time consuming!

Interruption-based design issues

Issue 1: How does the processor determine which device issued the interrupt?

Solutions:

3. **Daisy chain method:**
   - All I/O modules share a common interrupt request line.
   - When an interruption is detected an interrupt acknowledge (ACK) is sent.
   - ACK goes through the I/O modules until it gets to the requesting module.
   - Requesting module responds by placing a word on the data lines.
   - Word is either: Address of the I/O module or address of an adequate interruption handling technique.

Interruption-based design issues

Issue 2: If multiple interrupts have occurred, how does the processor decide which one to process?

Solutions:

1. **Multiple interrupt lines:**
   - Each line can have a predetermined priority;
   - Just choose the highest priority interrupt line
2. **Software poll/ Daisy chain:**
   - The order with each the modules are polled determines the priority in Software Poll.
   - The order of modules on a daisy chain determines their priority.

Direct Memory Access (DMA)

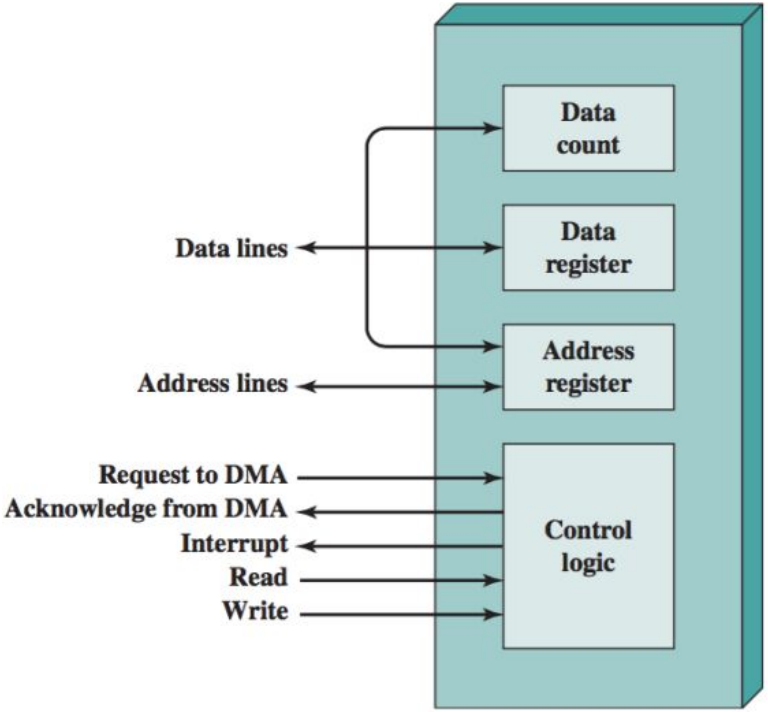
Copying data directly to memory, bypassing the processor.

- Memory accesses are performed by DMA module
- Unburdens the processor
- Combine with interruption scheme for optimum efficiency

Direct Memory Access

- An additional module on the system bus – DMA module.
- **DMA function**: the processor issues some commands to DMA module with the following information
  - Whether to READ or WRITE (using read/write control lines)
  - Address of I/O device involved (using data lines)
  - Starting location of the memory to read or write
  - The number of words to read/ write
- DMA module transfers **entire block of data** (bypassing the processor).
- Processor is only involved at the **beginning and ending** of the transfer as DMA module interrupts the processor after finishing transfer only.

Direct Memory Access



Reference:

1. Computer Organization and Architecture, by William Stallings, 10th edition.

Thank You!